

CHULETARIO DESARROLLO DE INTERFACES

MÉTODOS DE VENTANAS	
setLocationRelativeTo(null);	Centrar la pantalla
Dialogo dialogo = new Dialogo (this, true);	Para crear una instancia de la clase jDialog creada
setVisible(true);	Abrir un jDialog
setVisible(false);	Cerrar un jDialog
POP-UPS	
javax.swing.JOptionPane.showMessageDialog(this, "mensaje");	Para pop-up de mensaje. Devuelve un String.
javax.swing.JOptionPane.showInputDialog(this, "mensaje");	Pop-up para introducir datos. Devuelve el tipo de datos introducido.
javax.swing.JOptionPane.showConfirmDialog(this, "Mensaje");	Pop-up de confirmación. Devuelve un entero (0:Si, 1:No).
MÉTODOS DE CONTROL DE INFORMACIÓN	
Podemos manipular la información de los componentes que usamos para insertar datos, tales como el TextField, Spinner, ComboBox, etc.	
getText(); setText();	Obtiene el texto de un TextField o lo inserta en el mismo.
getItemSelected();	Obtiene el objeto seleccionado de un ComboBox. Es necesario casting
getValue();	Obtiene el valor seleccionado de un Spinner. Requiere casting
getDate();	Obtiene objeto tipo Date de la fecha seleccionada en un JDateChooser

setEditable(false);	Hace que un campo no pueda editarse.
MÉTODOS DE FECHAS	
LocalDate.now();	Obtiene la fecha actual
Instant.now().getEpochSecond();	Obtiene el tiempo UNIX. Almacenar en variable tipo long.
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy"); fecha.format(formatter);	Método para pasar de LocalDate a String
SimpleDateFormat formatter = SimpleDateFormat("dd/MM/yyyy"); formatter.format(fecha);	Método para pasar de Date a String
MÉTODOS DE TABLAS	
Para trabajar con tablas necesitamos asignarle un model del tipo DefaultTableModel. Para ello instanciamos el modelo de la siguiente forma: DefaultTableModel model = new DefaultTableModel(null, TABLE_HEADERS);	
jTable.setModel(model);	Asigna dicho modelo a una tabla.
addRow(Object[] row);	Añade una fila. Debemos de pasar un objeto tipo Array.
removeRow(index);	Elimina una fila según el índice.
getRowCount();	Devuelve un entero con el número de filas de la tabla. Ideal para recorrer con bucle.
getValueAt(2,0);	Obtiene el valor en función de la fila y columna. Útil para recorrer con bucles y manipular datos.
TableRowSorter: lo utilizamos para establecer un orden por defecto a la tabla. Para ello lo instanciamos de la siguiente forma: TableRowSorter sorter = new TableRowSorter<>(jTable.getModel());	
jTable.setRowSorter(sorter);	Asigna el sorter a una

	determinada tabla.
setSortKeys(Arrays.asList(new RowSorter.SortKey(2, SortOrder.ASCENDING), new RowSorter.SortKey(3, SortOrder.DECENDING)));	Establece el orden en función de la columna que queramos ordenar. Podemos añadir más columnas ya que lo que se le pasa al método es una lista de RowSorter.
sorter.sort();	Inicializa el sorter.
Caso particular: swing compara los elementos de las tablas a la hora de ordenarlos como si fuesen String, por lo que tendríamos un problema con los números ya que 10 se ordenaría antes que 9, por ejemplo. Para ello tenemos que utilizar un comparator.	
sorter.setComparator(1, (o1, o2) -> { Integer num1 = Integer.parseInt(o1.toString()); Integer num2 = Integer.parseInt(o2.toString()); return num1.compareTo(num2); });	Primero establecemos el comparator el cual nos servirá para ordenar los enteros y una vez establecido ordenamos la tabla con normalidad con el setSortKeys().
VALIDACIONES (LIBRERÍA EXTERNAL JVALIDATION)	
Los mensajes de validación aparecerán en el validationPanel. Tenemos que usar los grupos de validación para dicho panel: Validation group = validationPanel.getValidationGroup();	
group.add(TextField, new AbstractValidator<String>(String.class);	Establecemos un validador abstracto en función de las condiciones que pongamos, para un campo determinado.
problems.add("Mensaje de validación");	Añadimos condiciones.
group.add(jTextField1, StringValidators.REQUIRE_NON_EMPTY_STRING);	Validación por defecto. Solo valida que el campo no esté vacío y el mensaje ne inglés.
Para validaciones es mucho más útil combinar el AbstractValidator con expresiones regulares las cuales nos permiten tener mucho mayor control de lo que validamos.	
^[a-zA-ZáéíóúÁÉÍÓÚñÑ\\s]+\$	RegEx para validar que un

	campo solo tenga caracteres de texto.
<code>^[0-9]*\$</code>	RegEx para validar que un campo solo tenga números.
<code>^-?\d+\$</code>	RegEx para validar enteros tanto (positivos y negativos).
<code>jDateChooser1.getDateEditor().getUiComponent()</code>	Obtiene la fecha seleccionada del DateChooser y la valida.

TRABAJO CON IMÁGENES

Para trabajar con imágenes trabajaremos con sus rutas absolutas y relativas. También usaremos los Label para introducir imágenes en los mismos.

<code>jLabel.setIcon(new ImageIcon(getClass().getResource(PATH)));</code>	Establece una imagen en un Label
<code>setIconImage(new ImageIcon(getClass().getResource(PATH)).getImage());</code>	Cambia el icono de la app. El método se aplica al constructor de la clase principal.

LISTENERS

<code>TextField.getDocument().addDocumentListener(new javax.swing.event.DocumentListener());</code>	Método que permite ejecutar una acción en función de las actualizaciones del campo TextField.
---	---

LOOK AND FEELS

Para cambiar metemos todos en Combobox. Para ello tenemos que pasarle el modelo para editarlo, `DefaultComboBoxModel = new DefaultComboBoxModel();`

<pre>dcm = new DefaultComboBoxModel(); for (LookAndFeelInfo lfi : UIManager.getInstalledLookAndFeels()) { dcm.addElement(lfi.getName()); } dcm.addElement("Flat Light"); dcm.addElement("Flat Night"); jComboBox1.setModel(dcm);</pre>	Para acceder a la info de los laf, como el nombre. Podemos ponerlos dentro de un ComboBox o algun otro elemento para poder seleccionarlo.
--	---

<pre> String selected = (String) jComboBox1.getSelectedItem(); try { if ("Flat Light".equals(lookAndFeelName)) { UIManager.setLookAndFeel(new FlatLightLaf()); } else if ("Flat Night".equals(lookAndFeelName)) { UIManager.setLookAndFeel(new FlatDarkLaf()); } else { for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) { if (info.getName().equals(lookAndFeelName)) { UIManager.setLookAndFeel(info.getClassName()); break; } } } javax.swing.SwingUtilities.updateComponentTreeUI (this); } catch (Exception e) { e.printStackTrace(); } </pre>	Para establecer un look and feel tenemos que acceder a la opción seleccionada del ComboBox, comprobar que el LaF está instalado en el sistema y establecerlo con el setLookAndFeel y con updateComponentTreeUI.